

Python Scripting Integration - Help

Adroit Smart UI Designer

Overview

The Adroit Smart UI Designer supports Python as a third scripting language alongside Microsoft Visual Basic.NET and Microsoft C# (CSharp) for scripts in Graphic Form scripting, Script Spiders and Script DataSource. Python scripts are executed using Python.NET (pythonnet), which embeds the Python interpreter inside the .NET runtime and allows Python code to call .NET assemblies and interact with Smart UI components through the same INeed interfaces used by C# and VB.NET scripts.

This help covers configuring Python as your default scripting language, using the Script Editor for Python, managing Python packages, and writing Python scripts in the Designer.

How Python Scripts Are Executed

Unlike C# and VB.NET scripts, Python scripts are not compiled into a .NET assembly. The Script Engine uses Python.NET to:

- Initialize the Python runtime and connect to the Python 3.13 interpreter installed with the product.
- Create a Python execution scope and load referenced .NET assemblies using `clr.AddReference`.
- Execute the script source text within that scope.
- Call the script `main(ctx)` function, passing a context object with Application, Connection, Controller, Inputs, Outputs, and Control.

Default Python interpreter location:

`%ProgramFiles%\Adroit Technologies\Common\Python313\python.exe`

Optional: Set the `PYTHONNET_PYDLL` environment variable to point to a different Python DLL.

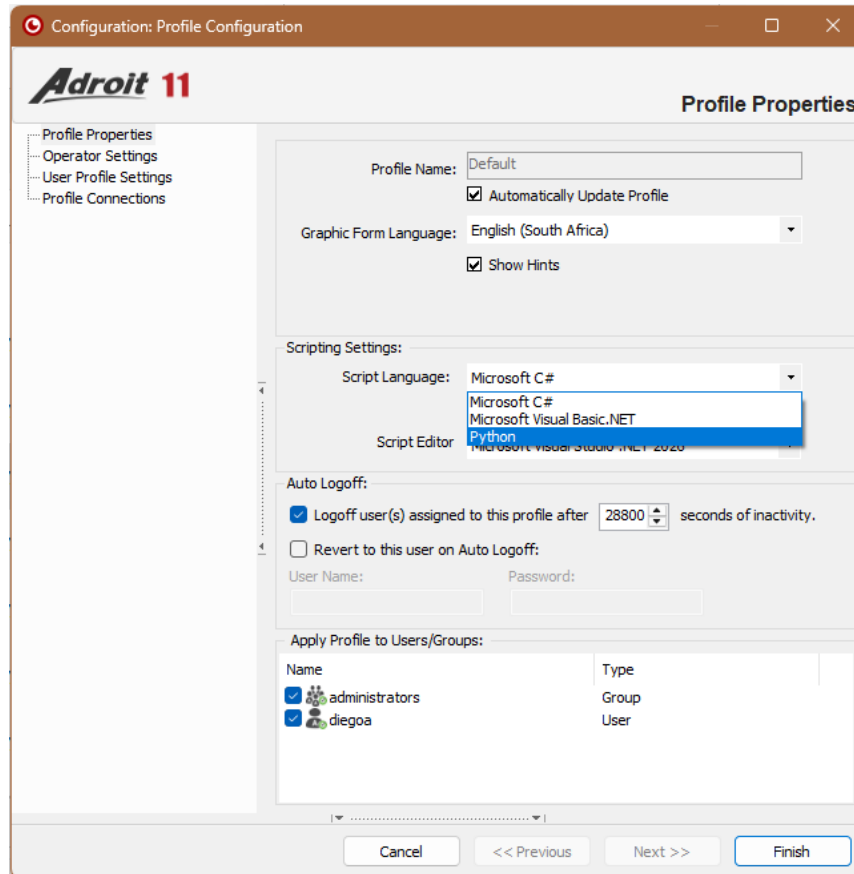
Setting Python as the Default Scripting Language (Profile DataSource)

The default scripting language for new scripts is determined by the profile associated with the logged-in user. Profiles are managed through the Profile DataSource in the Designer.

To set Python as the default:

- Expand the Profile DataSource and select (or create) the required profile.

- Configure the profile and in the Profile Properties, locate the Script Language setting.
- Select Python from the drop-down list (options: Microsoft Visual Basic.NET, Microsoft C#, Python).
- Save the profile configuration.

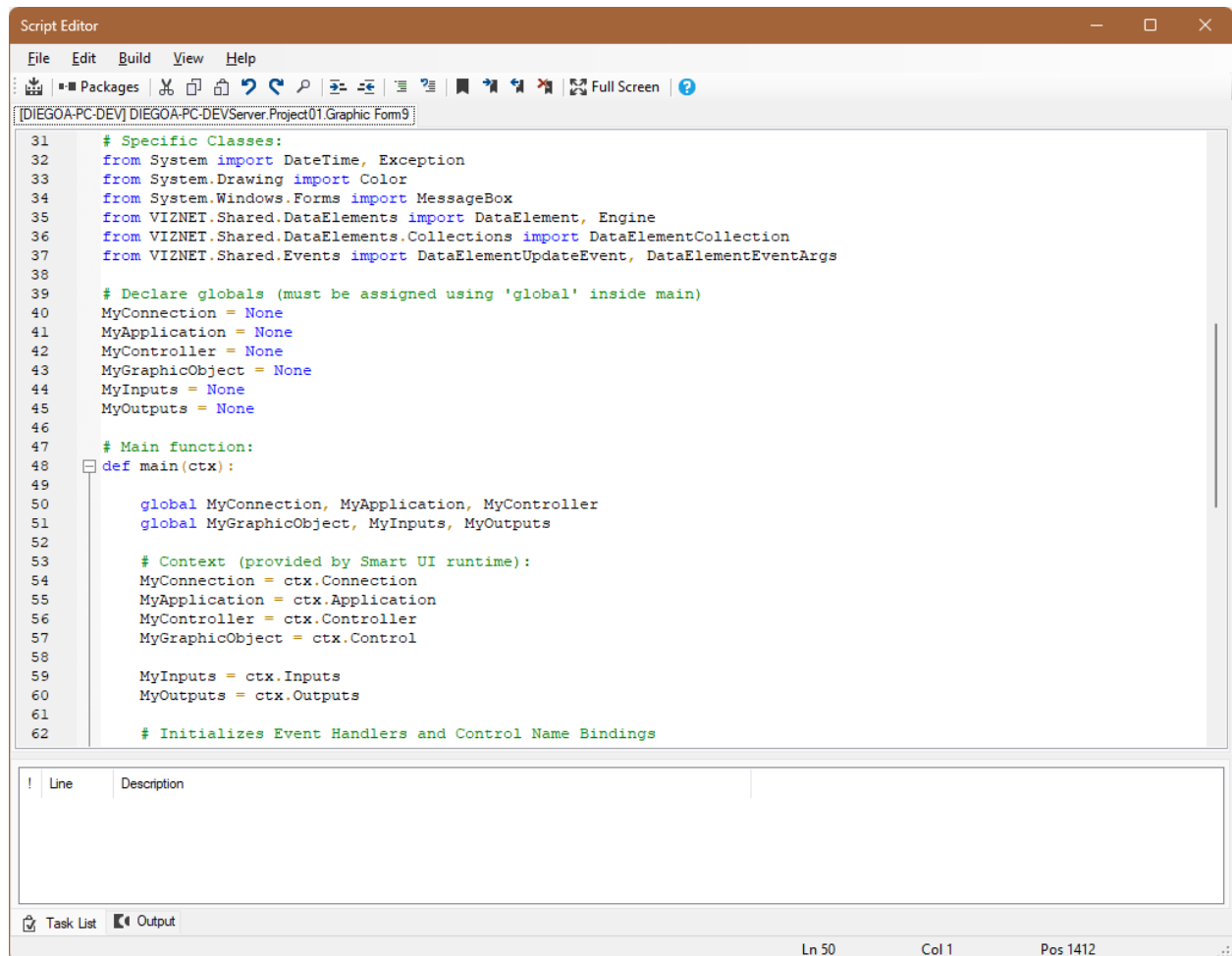


When a user assigned to this profile creates a new script, the Script Editor opens with Python selected and the default Python template loaded.

Note: The profile must also specify the Built-in script editor for scripts to open in the Designer Script Editor. You can override the profile default per script using Build > Languages in the Script Editor.

You may use other external Script Editors such as Visual Studio or Visual Studio Code to edit your Python scripts. This can be configured in the Script Editor setting in your profile.

Script Editor Changes for Python



Language Selection

Select Python from Build > Languages > Python. A tick mark indicates the active language. When switching to Python, you are prompted to reset the script source to the default Python template.

Syntax Highlighting and Indentation

- The editor switches to Python syntax highlighting.
- Indentation uses spaces instead of tabs (Python convention).
- The cursor is positioned at main(ctx) when a Python script opens.

Packages Instead of References

For C# and VB.NET, external assemblies are managed via Build > References. For Python scripts:

- References menu and toolbar button are hidden.
- Packages menu and toolbar button are shown instead.
- Solution Explorer and Class View panels are hidden (Python packages are environment-level, not per script).

.NET assembly references for Python are declared in script source using `clr.AddReference` statements in the default template.

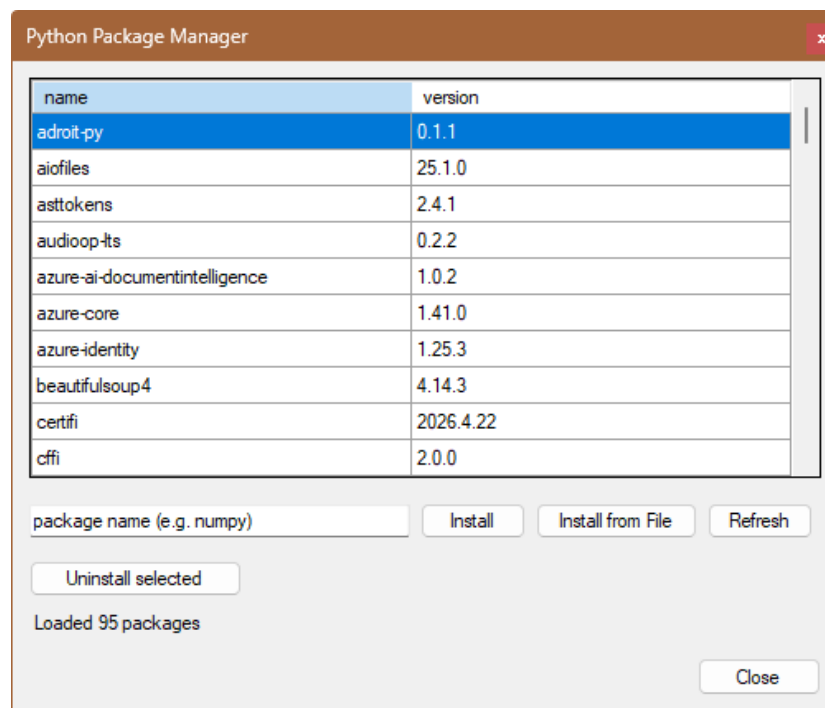
File Operations

The Script Editor supports .py files alongside .cs and .vb files for open and save operations.

Compile and Run

Python scripts are not traditionally compiled. Build > Compile validates the script is not at default template text. Build > Compile and Run executes via Python.NET. Every Python script must define `main(ctx)`. Runtime errors appear in the Output window.

Python Package Manager



The Python Package Manager dialog installs, views, and removes Python packages in the shared Python 3.13 environment. It uses pip for all operations.

Important: Unlike .NET references for C#/VB.NET, Python packages are NOT stored per script object. Packages are installed into the common Python environment and available to all Python scripts.

To open: Edit a Python script, then click Build > Packages or the Packages toolbar button.

View Installed Packages

- Open the Python Package Manager while editing a Python script.
- The upper list shows package name and version.
- Click Refresh to reload the list.

Install from Online Repository

- Enter the package name (e.g. numpy) or pip specifier (e.g. numpy==1.26.4) in the package name field.
- Click Install. Requires network access.
- After installation, import in scripts: `import numpy`

Install from Local File (Offline)

- Click Install from File and select a .whl (wheel) file.
- Ensure the wheel is compatible with Python 3.13 and your platform.

Uninstall a Package

- Select the package in the list and click Uninstall selected.
- Confirm the uninstall. The package is removed from the shared environment.

Writing Python Scripts

New Python scripts receive a default template containing:

- `import clr` and `clr.AddReference` calls for .NET and Smart UI assemblies.
- Python import statements for common namespaces.
- Global variables: `MyConnection`, `MyApplication`, `MyController`, `MyGraphicObject`, `MyInputs`, `MyOutputs`.
- `main(ctx)` entry point that receives the runtime context.
- `InitializeVIZNETComponents()` for component bindings and event handlers.

The main(ctx) Entry Point

Context Property	Description
ctx.Application	IObjectManager - application-level functions
ctx.Connection	Server connection and functions
ctx.Controller	INeed controller for spiders/controls by name
ctx.Control	Graphic object or host control
ctx.Inputs	Input data elements
ctx.Outputs	Output data elements

Accessing .NET from Python

- Add reference: `clr.AddReference("AssemblyName")`
- Import namespace: `import Namespace.Path` or `from Namespace.Path import ClassName`

The default template pre-loads commonly used Smart UI and System assemblies.

Property Grid Integration

Components and event handlers can be added to Python scripts from the Property Grid when working with Script Engine spiders. Code is automatically inserted into `InitializeVIZNETComponents()`. The default template must contain this function.

Differences: Python vs C#/VB.NET

Feature	C# / VB.NET	Python
Execution	Compiled to .NET assembly	Interpreted via Python.NET
Entry point	Main() method	main(ctx) function
External libraries	.NET References (per script)	Python packages (shared) + <code>clr.AddReference</code>
Indentation	Tabs (default)	Spaces (default)
Solution Explorer	Shown	Hidden for Python
Compile	Full compilation	Runtime execution, no traditional compile

Help and Learning Resources

Script Editor Help menu links:

- Official Python Documentation - <https://docs.python.org/3/>
- Python Tutorial (W3Schools) - <https://www.w3schools.com/python/>
- Adroit Knowledge Base - <https://adroittechnologiesautomation.com/>

Troubleshooting

- `main(ctx)` not defined: Use Build > Languages > Reset Script Source to restore the default template.
- Import error for package: Verify package in Python Package Manager; reinstall if needed.
- Package install fails: Check network for online installs; use Install from File with compatible .whl (Python 3.13).
- .NET type not found: Add `clr.AddReference("AssemblyName")` before importing.
- Wrong language selected: Confirm Python under Build > Languages or set Python in Profile DataSource.